Merge Sort

We break the array into smaller parts, sort them, and then merge them back.

Example :

[8, 3, 5, 2]

Divide → [8,3] and [5,2]

Divide → [8],[3] and [5],[2]

Merge → [3,8] and [2,5]

Final Merge → [2,3,5,8]

🔹 **2. Key Concept**

Introduce:

- Divide → Split array into halves
- Conquer → Sort each half recursively
- Combine → Merge sorted halves

3. Dry Run (Important for Students)

Take example:

arr = [4, 1, 3, 2]

[4,1,3,2]

→ [4,1] [3,2]

→ [4] [1] [3] [2]

→ [1,4] [2,3]

→ [1,2,3,4]

4. Code (Python – Clean & Easy)

```python
def merge_sort(arr):
    # Base case
    if len(arr) <= 1:
        return arr

    # Step 1: Divide
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Step 2: Merge
    return merge(left, right)
```

```python
def merge(left, right):
    result = []
    i = j = 0

    # Compare elements and merge
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
```

```python
    # Add remaining elements
    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result


# 🔹 Example Usage
if __name__ == "__main__":
    arr = [8, 3, 5, 2, 9, 1]
    sorted_arr = merge_sort(arr)
    print(sorted_arr)
```